Deconstructing *Minecraft* (Java Edition, Survival mode)

Anshul Reddy Thodupunuri · ISEP

Why this game. Minecraft is a fully 3D, fully mutable world filled with autonomous agents (mobs), which makes it a good fit for the "space / player / agents / behaviours" breakdown of the course. Its real save format is publicly documented, so I can check the data structures I extrapolate against what the developers actually did. It also has a long VR history (the Vivecraft mod, and a former native VR mode on Bedrock), which keeps the analysis close to the VR/AR focus of the course.

1 Game design components

Space	Discrete 3D voxel grid of 1 m³ blocks, procedurally generated from a seed. Practically infinite horizontally (world border at ±30 million blocks), bounded vertically (Y −64 to 320). Partitioned into 16×16 *chunks* that are only simulated near players. Three linked dimensions: Overworld, Nether (1 block = 8 in the Overworld, a travel shortcut) and End. **Every block is editable**, so the space is also the main resource. Entities move continuously (float positions, box colliders) on top of the discrete grid. Each block also carries a **light level** (0–15).
Time	Fixed simulation step of 20 ticks/s. A 24,000-tick (20 min) day/night cycle; "random ticks" grow crops and spread grass.
Player	First-person avatar with health (20 half-hearts), hunger + hidden saturation, XP, a 41-slot inventory and a respawn point. No class, no stats tree: the player's power is entirely in their *items*.
Agents	**Passive** (cow, sheep, chicken: food, wool, breeding) · **Neutral** (wolf, enderman, bee: hostile only if provoked) · **Hostile** (zombie, skeleton, creeper, spider) · **Social** (villagers: professions, trading, schedules) · **Bosses** (Ender Dragon, Wither). Non-living "agents" also act on the world: flowing water/lava, fire spread, gravity blocks, redstone circuits.
Actions	Move (walk, sprint, jump, sneak, swim, elytra flight) · mine (speed depends on tool tier) · place · craft (2×2 / 3×3 grid recipes), smelt, enchant, brew · attack / block with shield · use / interact (eat, sleep, trade, breed, tame, ride) · wire redstone logic.
Behaviours	Mobs run a **prioritised goal list** (e.g. zombie: attack player > break door > wander > look around) with A* pathfinding on the block grid and limited sensing range. Villagers use a richer "brain" with sensors, memories and a daily schedule (work, meet, sleep). Signature behaviours create the game's tension: the *creeper* silently approaches, hisses for 1.5 s and explodes, **destroying terrain**; the *enderman* turns hostile when you look it in the eyes and steals blocks. Spawning is driven by the space itself: hostile mobs spawn at block light 0.
Rules & goals	No mandatory goal. Implicit goals: survive, climb the tool tiers, build. The soft "ending" is defeating the Ender Dragon; ~120 optional *advancements* act as hints. Death drops your items at the death spot.
Chance vs skill	**Chance:** world seed, ore placement, loot tables, spawns, drop rates, enchantment rolls. **Skill:** combat timing (attack cooldown), navigation, resource planning, architectural creativity, redstone engineering.

My reading: the central design idea is that **space, resources and agents are the same system**. Light decides where enemies appear, so building (placing torches, walls) is also defence; creepers attack the space itself, so they threaten what the player made, not just their health bar.

2 Game loop

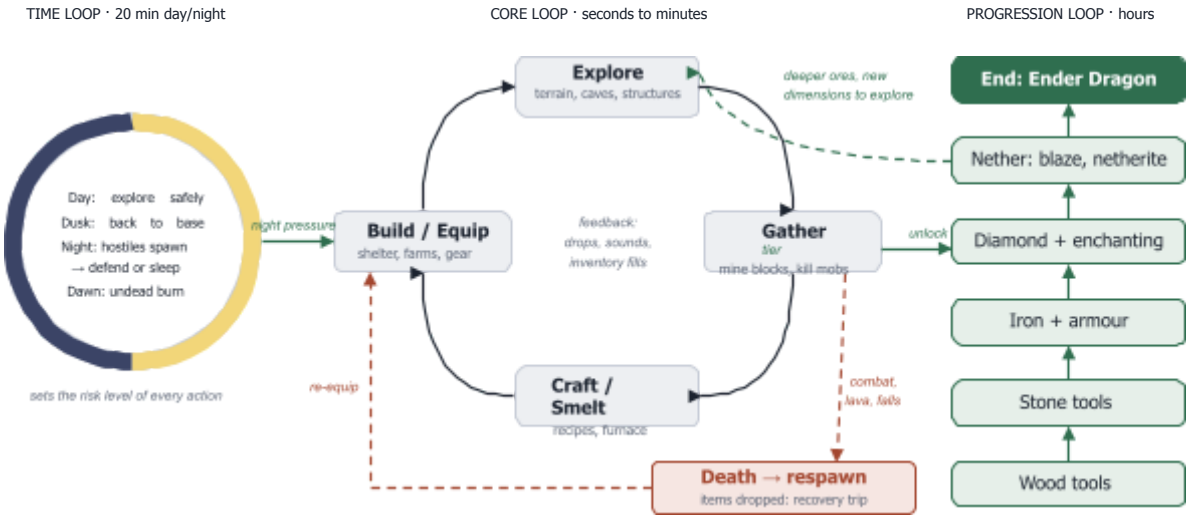


Figure 1 — Three nested loops: the day/night cycle modulates risk, the core loop produces resources, and the progression ladder unlocks new zones that feed exploration.

The core loop is short and always gives feedback (block-break sound and particles, item pickup, a full inventory). The **day/night cycle is the pacing device**: it forces the player to switch from "explore" to "build" mode every ten minutes, which is how a sandbox without goals still produces rhythm. Death is cheap in time (instant respawn) but costly in items, which turns a loss into a new short-term goal: the recovery trip.

3 Data structures for a full game state

Written as C# classes (close to how I would do it in Unity). Two principles make it scale to an infinite world: **seed + delta** (only chunks that were generated are saved; the rest is regenerated deterministically from the seed) and **palette compression** (a 16^3 section usually holds only a few block types, so each block stores a small index into a palette instead of a full block name).

```
// Root object = one save folder
class WorldSave {
  WorldMeta meta;
  Dictionary<DimensionId, Dimension> dimensions;
  Dictionary<Guid, PlayerData> players;
}
class WorldMeta {
  long seed;     // regenerates untouched terrain int
  dataVersion;  // for save migration
  long gameTime, dayTime; // ticks (20/s)
  Difficulty difficulty; bool hardcore;
  Weather weather;        // rain/thunder +
  timers Vector3Int worldSpawn;
  Dictionary<string,string> gameRules;
  bool dragonKilled;      // progression flags
}
class Dimension {         // Overworld, Nether, End
  // only generated chunks are stored, grouped
  // in region files of 32x32 chunks
  Dictionary<Vector2Int, Chunk> chunks;
}
class Chunk {    // 16 x 384 x 16 column
  Vector2Int pos; GenStatus status;
  Section[] sections; // 24 cubes of
  16x16x16 List<BlockEntity> blockEntities;
  List<Entity> entities;
  List<ScheduledTick> pendingTicks; // water,
  redstone long inhabitedTime; // local difficulty
}
class Section { // palette compression
  List<BlockState> palette; // unique states
  ulong[] packedIndices; // 4096 ids, few bits
  byte[] blockLight, skyLight; // 4 bits / block
  byte[] biomes;
}
```

```
class BlockState {
  string blockId;         // "oak_stairs"
  Dictionary<string,string> props; //
  facing=north
}
class BlockEntity {       // blocks with extra
  data Vector3Int pos; string type;
  ItemStack[] items;      // chest, furnace slots
  Dictionary<string,object> data; // smelt
  progress
}
class Entity {  // every mob and dropped item Guid
  uuid; string type;
  Vector3 pos, velocity; Vector2 rotation;
  float health; int fireTicks; bool
  onGround; List<StatusEffect> effects;
  AIMemory memory;        // target, home,
  angerTime Dictionary<string,object> extra; //
  trades, owner
}
class PlayerData : Entity {
  ItemStack[] inventory = new ItemStack[41];
                // 36 main + 4 armour +
  offhand ItemStack[] enderChest = new
  ItemStack[27]; int food; float saturation,
  exhaustion;
  int xpLevel; float xpProgress;
  GameMode mode; Vector3Int? bedSpawn;
  HashSet<string> advancements, knownRecipes;
  Dictionary<string,int> statistics;
}
class ItemStack {
  string itemId; int count;
  int damage;    // durability used
  Dictionary<string,int> enchantments;
  string customName;
}
```

What is **not** saved is as interesting as what is: the current A* path of a mob, particles and sound are recomputed after loading; only the AI's *memory* (target, home, anger timer) is persistent. Comparing with the real format, Minecraft stores almost exactly this as NBT (a binary tree of named tags) inside region files (`.mca`, 32×32 chunks each), plus `level.dat` for the world meta and one file per player.

4 Gamification frameworks

MDA — Mechanics → Dynamics → Aesthetics (Hunicke, LeBlanc, Zubek)

Mechanics (rules, data)	Dynamics (emergent behaviour)	Aesthetics (player experience)
Editable blocks, crafting recipes	Players build bases, monuments, whole cities	**Expression** (primary)
Seeded generation, caves, loot tables	Strip-mining vs cave-diving strategies; seed sharing	**Discovery** (primary)
Spawning at light 0, creepers, hunger	Lighting up areas, building walls, farming food	**Challenge**, tension at night
Redstone, mob AI rules	Automatic farms, machines, computers built in-game	Expression + Challenge (engineering)
Multiplayer, villager trading	Shared builds, player economies, also griefing	**Fellowship**
Calm music, peaceful mode, slow crops	Relaxing "chill building" sessions	**Submission** (pastime); Narrative is weak

Orientation: MDA looks at the game as a *system* from the designer's side. It explains the fun as emergent and pushes me towards **tuning mechanics** (ore rarity curves, spawn rules, light thresholds) to obtain a target dynamic.

6D — Werbach & Hunter

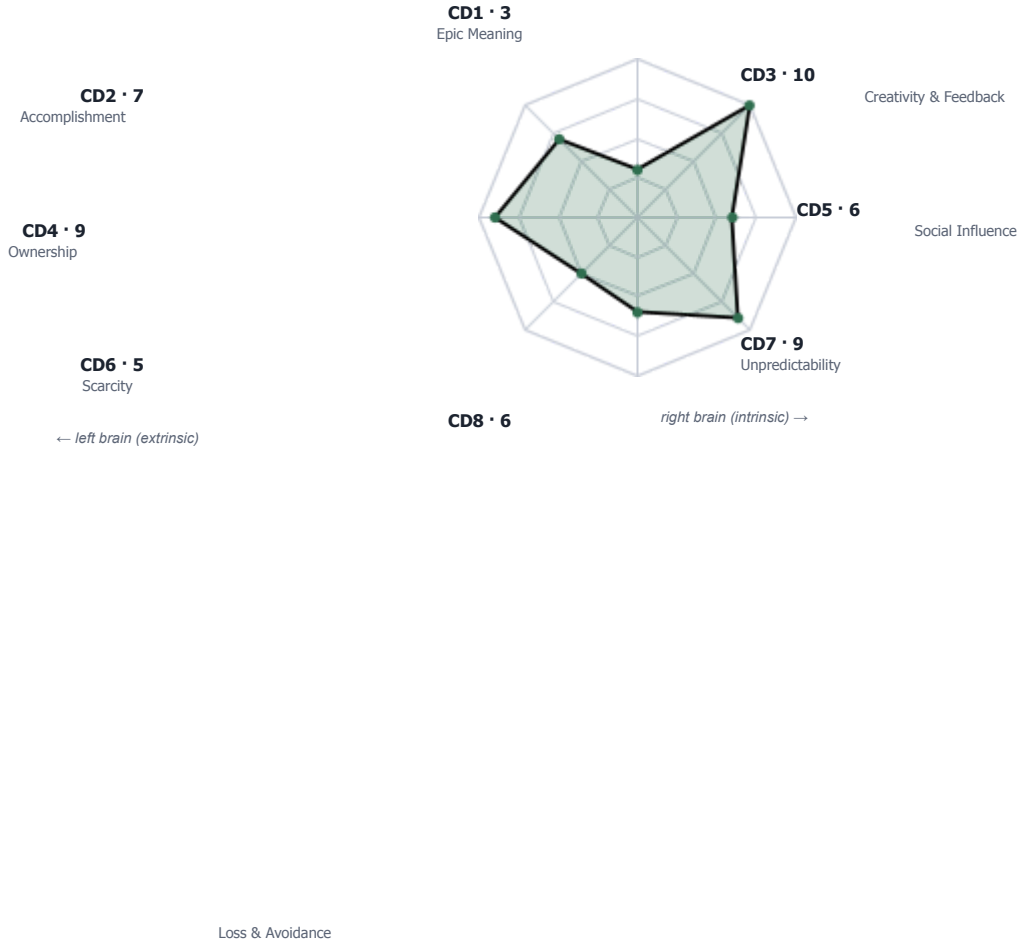
6D was built to gamify *non-game* activities, so applying it to a game forces a business/product reading:

1.	**Define objectives**	Long-term retention, community and content creation, sales (Realms servers, Marketplace, Education Edition).
2.	**Delineate behaviours**	Return often, play with friends, create and share builds/maps. Metrics: session length, return rate, server population.

3.	**Describe players** Bartle: Explorers (caves, seeds), Builders/Achievers (advancements, speedruns), Socializers (servers), Killers (PvP servers). Very wide age range.
4.	**Devise loops** Engagement loop = motivation → action → feedback (mine → drop + sound → craft); progression loop = tool tiers and advancements.
5. Don't forget fun	The fun is intrinsic (building, exploring); nothing is gamified on top of a boring task.
6.	**Deploy tools** Points = XP bar; badges = advancements; progress bars = XP, block-breaking animation. **No leaderboards in vanilla**: servers add them.

Orientation: 6D shows how **little classic PBL** (points, badges, leaderboards) vanilla Minecraft uses, and points to player segmentation and measurable behaviours. Its design result is product-level: add social and progression tools for each player type, which is exactly what Realms, servers and the Marketplace do around the game.

Octalysis — 8 core drives (Yu-kai Chou)



CD3 Creativity (10) is the core of the game; **CD4 Ownership (9)** comes from "my world, my base, my tamed wolf"; **CD7 Unpredictability (9)** from procedural terrain and loot. **CD2 Accomplishment (7)**: tool tiers, enchanting, advancements. **CD8 Loss (6)**: dropped items, creepers destroying builds, Hardcore mode. **CD5 Social (6)**: strong in multiplayer and community culture, weak in solo play. **CD6 Scarcity (5)**: rare diamonds, crop wait times. **CD1 Epic Meaning (3)**: almost no story or purpose beyond "beat the dragon".

The shape leans **right-brain** (intrinsic: 3, 5, 7), which explains why players stay for years without rewards, and mixes **white hat** (empowering, 2–3) with enough **black hat** (6–8) at night to create urgency.

Orientation: Octalysis looks at the player's *motivation* and immediately shows the **gap**: Epic Meaning.

Figure 2 — My Octalysis scoring of Minecraft Survival (0–10).

Three frameworks, three different design answers. MDA says "tune the mechanics" (system view); 6D says "track behaviours and add tools per player type" (business view); Octalysis says "strengthen the weakest drive, CD1" (motivation view). The same game is read as a simulation, a product, and an emotional profile.

5 Strengths, weaknesses and a proposed modification

Strengths

- One consistent rule set (every block obeys the same physics) produces huge emergent depth.
- Very strong ownership and creativity drives, which are intrinsic and long-lasting.
- Endless replayability through seeds; modding and servers extend the game.
- Low-poly blocky style: cheap to render and readable, a big advantage in VR.

Weaknesses

- Weak onboarding: no tutorial, recipes are often learned from a wiki.
- Weak CD1: once the player has diamonds or killed the dragon, there is a "what now?" drop-off.
- Solo play lacks social drive (CD5).
- Black-hat losses (a creeper blowing up hours of building) can feel unfair and make players quit.

Modification — "Village Guardian" (targets CD1 Epic Meaning, supported by CD3 and CD5). Villages start damaged and threatened by illager raids. Villagers post optional *requests* on a village board (a wall, a farm, a lighthouse, a bridge), but **how** to build them is left completely free: the game only checks function (e.g. "the farm feeds 5 villagers", "no mob can enter"). Each fulfilled request makes the village grow (new professions, new trades, villagers visibly using what you built) and is written into a **village chronicle** book that names the player. In multiplayer, the chronicle credits every contributor. **Why it works:** it gives the player's creativity a purpose bigger than themselves (CD1) without imposing a story, and turns the "what now?" late game into "who needs me?". **Risk to watch:** extrinsic rewards can crowd out intrinsic motivation (over-justification), so requests stay optional, give no power-ups, and reward with *meaning* (growth, recognition) rather than loot.

What I take into my own project (Part 2). (1) Make agents react to a *property of the space* (like light → spawning) instead of to hard-coded triggers, so the player's building choices change agent behaviour. (2) Add a slow time loop on top of the core loop to create rhythm and switch the player between modes. (3) Keep the save state minimal: seed + changes + agent memory, never transient data like paths.

Part 2: Game Design Document — *Warren*

Gamification of Homework 3 (EcoSim: Predator & Prey) · Unity project `H4ThodupunuriAnshul` · namespace `ThodupunuriAnshul.EcoSim`

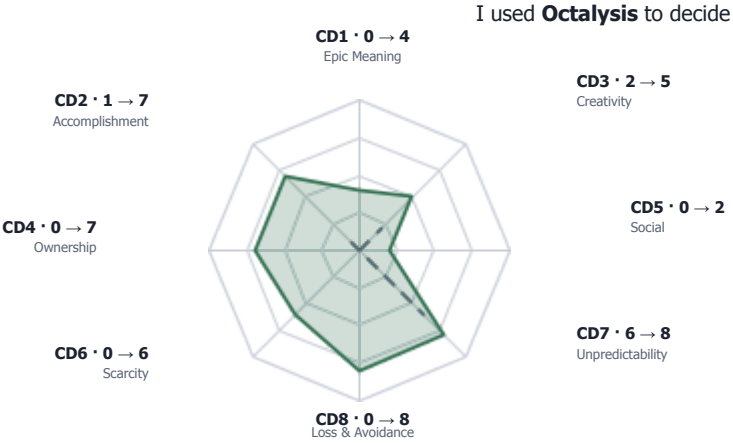
Pitch. You are one rabbit inside the Homework 3 ecosystem. Eat, breed, hide, and keep your **warren** (your descendants) alive for 7 nights while foxes stalk and wolf packs hunt. When your rabbit dies, you continue as one of its children. The predators, the herds and the grass are the *same agents* as in Homework 3: the simulation is the opponent.

A Starting point: Homework 3 reviewed with the game design components

Component	Homework 3 — EcoSim (a simulation)	Homework 4 — Warren (a game)
Space	50×50 m flat arena with walls; agents use distance checks, no physics.	Same arena + **burrows**: zones where rabbits are hidden. Agents react to this property of the space (idea taken from my Part 1 analysis: light → spawning in Minecraft).
Time	Real time, speed buttons ×1–×4; plant growth ticks, census every 3 s.	**Day/night cycle**: 60 s days, nights from 25 s growing +5 s per day. 7 days.
Player	None. An observer with an orbit camera.	One **possessed rabbit**; on its death control passes to an heir (lineage).
Agents	Grass, rabbit, deer, fox, wolf, with an inheritance tree of 3 levels.	Same agents. Predators get sharper every night; warren rabbits follow the player.
Actions	Camera and time speed only.	Move, sprint, eat, mate, thump, dig.
Behaviours	Priority FSM (survive > eat > reproduce > wander), herd alarm, pack call, fox stalking.	+ run to burrows, hidden prey ignored, alarm shared by all herbivores (used by Thump).
Rules & goals	None: it runs forever.	Survive 7 nights with at least one warren rabbit. Score, stars, best run.
Chance / skill	Chance only (spawns, litters, seeds).	Skill: routes, meal timing, stamina, burrow placement. Chance: AI decisions, litters, heirs.

Diagnosis. Homework 3 already had rich *dynamics* (herds fleeing together, packs converging, population cycles) but no player, no goal and no stakes. In Octalysis terms it only activated CD7 (curiosity about what emerges). The design job is to put the player **inside** the system, on the weakest side: the prey.

B Gamification framework: Octalysis to choose, MDA to build



I used **Octalysis** to decide *which motivations* to target, then **MDA** backwards (aesthetic → dynamic → mechanic) to decide *what to code*.

Primary drive: CD8 Loss & Avoidance. Being prey is the fantasy. Every meal freezes you for 1.5 s, sprinting costs stamina, and a death costs a generation. To keep the loss fair, **CD4 Ownership** and **CD2 Accomplishment** balance it: your rabbit has a marker, your warren wears a gold tag and follows you, and the warren size unlocks abilities and stars.

CD7 comes for free from the Homework 3 AI. I protected it on purpose: a stalking fox is never announced, the HUD only says *"You feel watched…"*. **CD6 Scarcity**: stamina, cooldowns, grass that must regrow, nights that shorten

foraging.

Balance check. The profile is black-hat heavy (6, 7, 8), which creates urgency but tires players. So a run lasts about 12 minutes, and the lineage mechanic turns most deaths into "continue as your child" instead of "game over". **CD5 Social** stays weak: this is the next improvement (a shared-seed leaderboard).

Figure 3 — Octalysis profile before (dashed) and after (green).

Target aesthetic (MDA)	Dynamics wanted	Mechanics implemented
Challenge: the tension of being prey	Risky foraging, last-second dashes to a burrow	Eat lock 1.5 s (move = drop the meal), 4 s stamina, predator senses ×1.35 at night, +2.5 % speed per day, new predators each night
Fantasy: being a rabbit	Hiding, warning the others, zig-zag escapes	Burrows (the body sinks in), Thump (real rabbit behaviour, re-uses the herd alarm), hopping animation
Narrative: an emergent family story	"My great-grandchild finished the run"	Lineage transfer to the best adult heir, generation counter, end-screen summary
Fellowship (light)	Protecting your followers	Warren rabbits follow the leader; Thump sends them to burrows

C Game loops

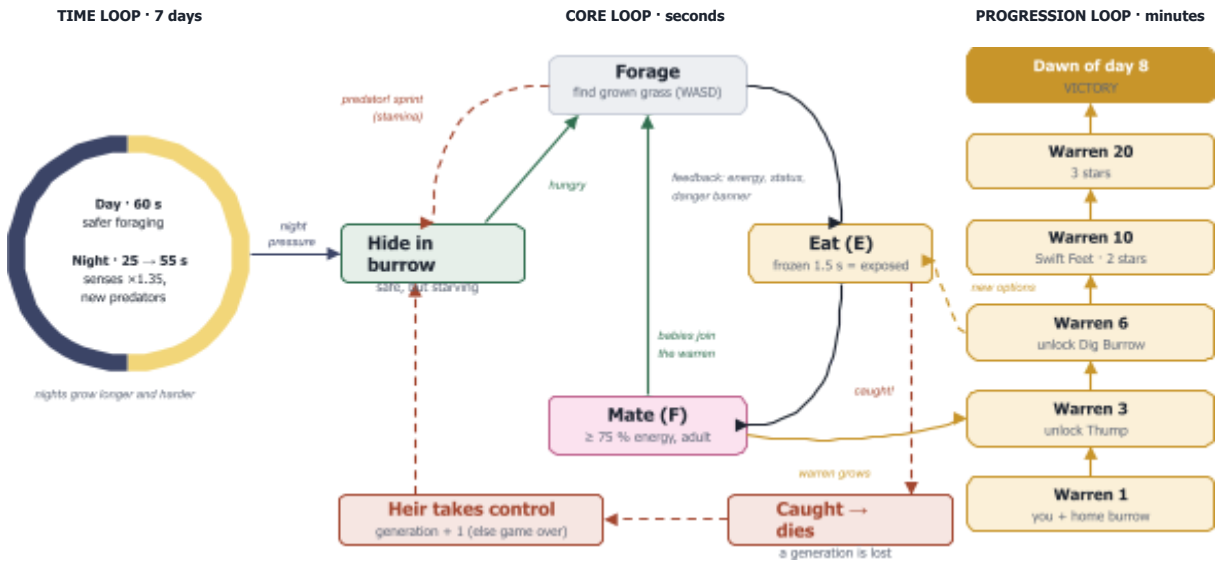


Figure 4 — Core loop (forage / eat / mate / hide), time loop (day/night pressure), progression loop (warren milestones), and the lineage branch on death.

D Rules and content

Ability	Unlock	Key	Effect
Thump	warren 3	Space	Warren rabbits within 12 m flee to burrows from the nearest predator. Cooldown 8 s.
Dig Burrow	warren 6	B	New safe zone (max 4, 6 m apart). Costs 20 % energy, cooldown 20 s.
Swift Feet	warren 10	passive	+15 % speed.

1	25 s / ×1.35	1	×1.00	
2	30 s / ×1.40	1	×1.025	×1.04
3	35 s	2	×1.08 / ×1.46	×1.050
4	40 s	2	×1.12 / ×1.51	×1.075
5	45 s	3	×1.16 / ×1.57	×1.100
6	50 s	3	×1.20 / ×1.62	×1.125
7	55 s	4	×1.24 / ×1.67	×1.150

Win: reach the dawn of day 8. **Lose:** no warren rabbit left. **Score** = 100/day + 20/peak warren + 5/birth + 50/extra generation (+500 on victory). Stars on victory: 1, 2 at warren 10, 3 at 20.

Onboarding: day 1 starts with only 2 foxes and 1 wolf, the founder spawns inside the home burrow, and the first message explains E / F / burrows. Every value in these tables is a field of `DifficultySettings` / `ProgressionSettings` on the GameManager, so balancing happens in the Inspector.

E Translating the design into Unity: foundations first, then detail

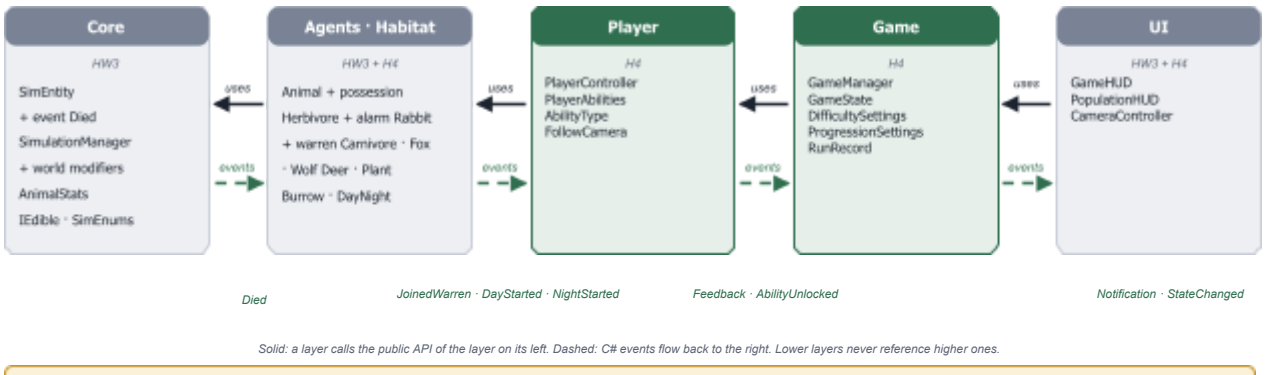


Figure 5 — Namespaces and dependencies. Each folder of `Assets/Scripts` is a sub-namespace of `ThodupunuriAnshul.EcoSim`.

#	Step	Done when
1	**Refactor agents for possession**	`Animal` accepts commands (move, eat, mate) and keeps its own rules; AI and player share `FindFood(range)` and `MeetsMatingRequirements`.
2	**Minimal loop:** PlayerController + FollowCamera	I can move a rabbit, eat, sprint with stamina, and be eaten.
3	**GameManager** state machine + GameHUD	Title → Playing → Paused → GameOver, restart by scene reload.
4	**Burrows** (Habitat)	Hidden rabbits are ignored by predators; AI rabbits flee into them.
5	**Warren + lineage**	Babies inherit membership; on death the best heir is possessed.
6	**Day/night + difficulty curve**	Events drive world modifiers and night spawns; the lighting changes.
7	**Abilities + unlocks**	Thump re-uses the alarm moved to `Herbivore`; Dig spawns the Burrow prefab.
8	**Score, stars, best run**	RunRecord saved as JSON; end screen and title show it.
9	**Detail & feel**	Marker, gold tags, sinking into burrows, danger banner, "you feel watched", toasts, builder generating everything.

F Key technical decisions

- **Possession instead of a separate player class.** A `PlayerRabbit` class would have duplicated movement, energy, eating and mating. Instead `Animal.Update` chooses between two "brains" (AI or player commands), so the player is always subject to the same rules as every rabbit, and control can jump to any heir.
- **Refactoring before adding.** The deer herd alarm moved up to `Herbivore` (generic `AlertNearby<T>`); Thump re-uses it in a few lines. `SenseFood()` became `FindFood(range)`, so the AI senses and the player's reach share one query.
- **Hidden prey through an existing rule.** Burrows only change `CanBeEaten`. The predators did not need any new code.
- **Loose coupling with events.** `SimEntity.Died`, `Rabbit.JoinedWarren` and `DayNightCycle.DayStarted` let the GameManager react without the simulation knowing it exists. Static events are unsubscribed in `OnDestroy`, because a restart reloads the scene.
- **Data-driven balancing.** `[Serializable]` settings classes on the GameManager, and the best run stored as JSON (`JsonUtility` + `PlayerPrefs`).
- **Everything generated.** The editor builder creates all assets and wires every reference (`SerializedObject`), so the project rebuilds from scripts alone.